ASIR

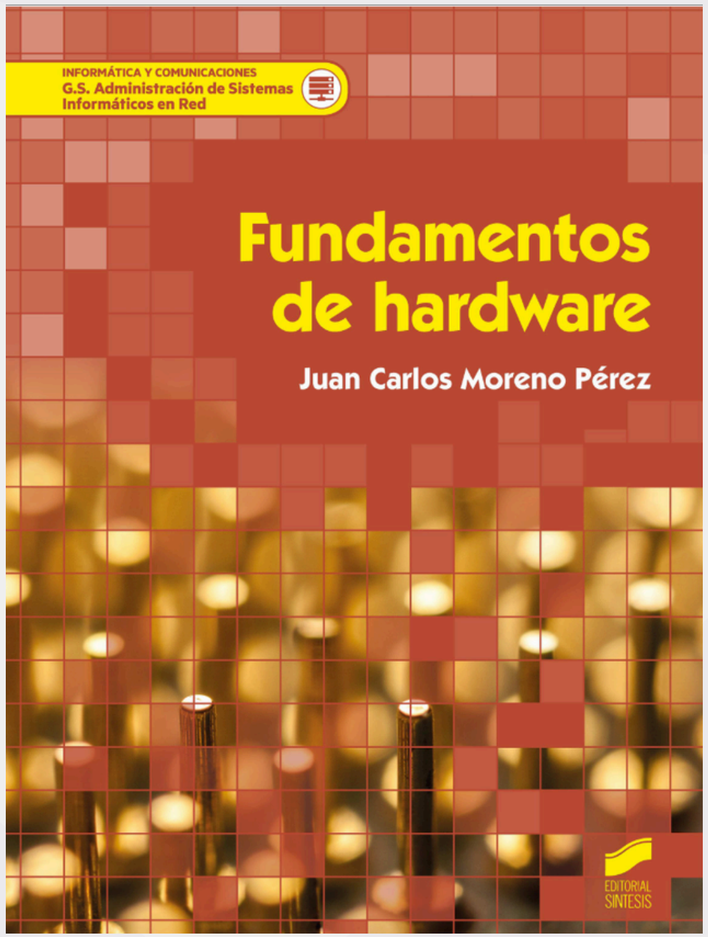
# HARDWARE FUNDAMENTALS

30/09/2026

**Teacher:**
Begoña Rodríguez Ferreras

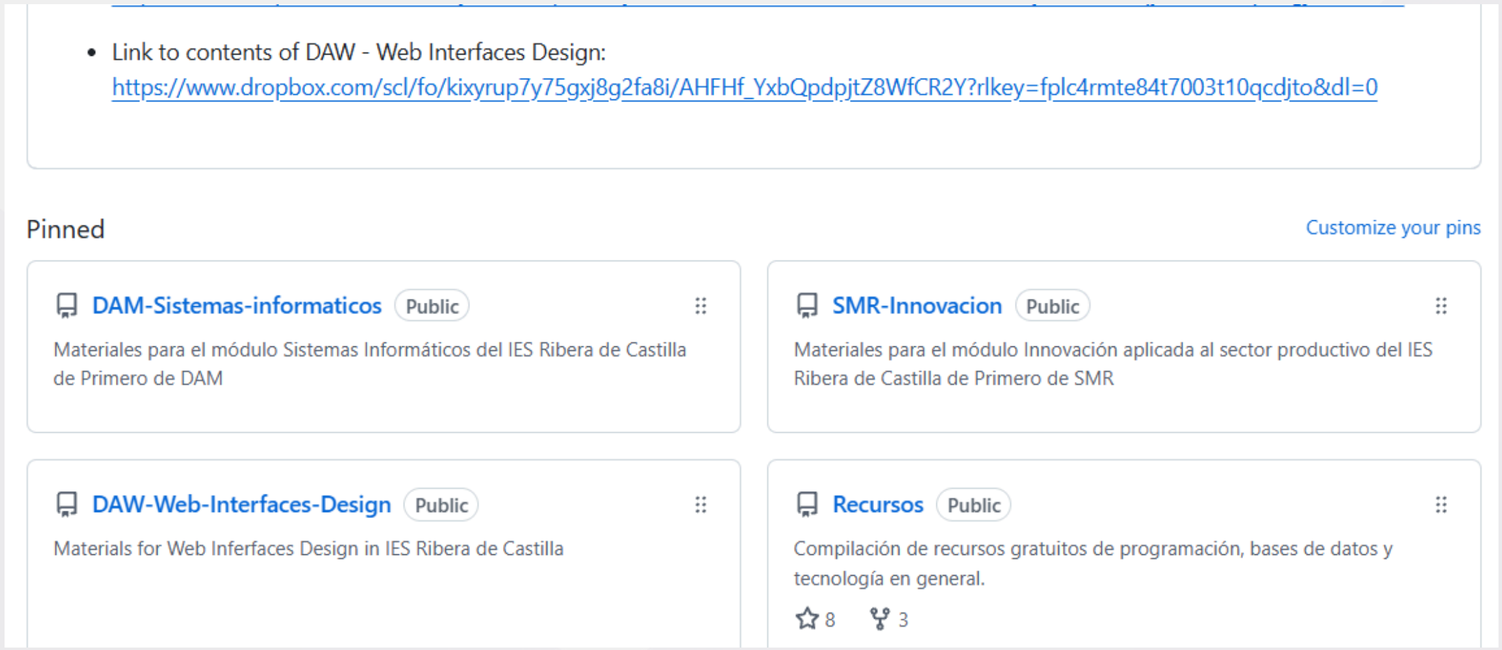
**begona.rodfer.1@educa.jcyl.es**

# BOOK


INFORMÁTICA Y COMUNICACIONES
G.S. Administración de Sistemas Informáticos en Red
Fundamentos de hardware
Juan Carlos Moreno Pérez
EDITORIAL SINTESIS

# MATERIALS

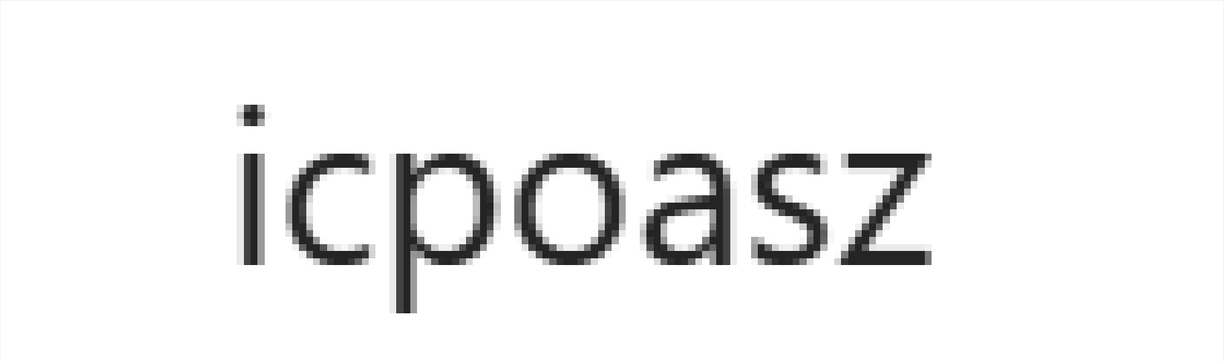
GitHub: https://github.com/BegoRodriguez/DAW-Web-Interfaces-Design
Dropbox: https://www.dropbox.com/scl/fo/kixyrup7y75gxj8g2fa8i/AHFHf_YxbQpdpjtZ8WfCR2Y?rlkey=fplc4rmte84t7003t10qcdjto&dl=0



# TEAMS

I have created a classroom team for communication and project and activities delivery on Teams.

This is the code:



# COMPUTER ARCHITECTURES: FUNCTIONAL ELEMENTS OF A COMPUTER

**Objetives:**

- Explore computing from the perspective of computer hardware and systems.
- Understand the classical Von Neumann architecture, including its functional units and subsystems.
- Learn about the architectures currently used in mobile devices.
- Understand the concept of System-on-Chip (SoC).
- Identify emerging processor architectures, including neuromorphic chips and AI-oriented architectures.

# COMPUTER ARCHITECTURES: FUNCTIONAL ELEMENTS OF A COMPUTER (II)

- Introduction
- Computer Architecture. Functional Units and Subsystems
  - Memory
  - Input/Output Subsystem
  - The ALU (Arithmetic Logic Unit)
  - The Control Unit
- SoCs. Mobile Processors
- New Microprocessor Architectures: Neuromorphic Chips
- Artificial Intelligence in Processors

# THINK ABOUT IT FOR A MOMENT...

Computers are part of everything we do. We use them without even noticing: surfing the web, writing, emailing, gaming, watching content. So let’s start with a simple but powerful question: what is a computer, really? An appliance? A toy? A typewriter? Something more?



# AUTOEVALUATION

A computer is...

... a machine for writing documents
... a calculator
... a general purpose machine that works with given data and provides results
... a machine to see TV

# AUTOEVALUATION (II)

A computer is...

... a machine for writing documents
... a calculator
**... a general purpose machine that works with given data and provides results**
... a machine to see TV

# COMPUTER DEFINITION

A computer is an electronic, programmable device capable of receiving data as input, processing and storing it according to a set of instructions, and producing information as output.

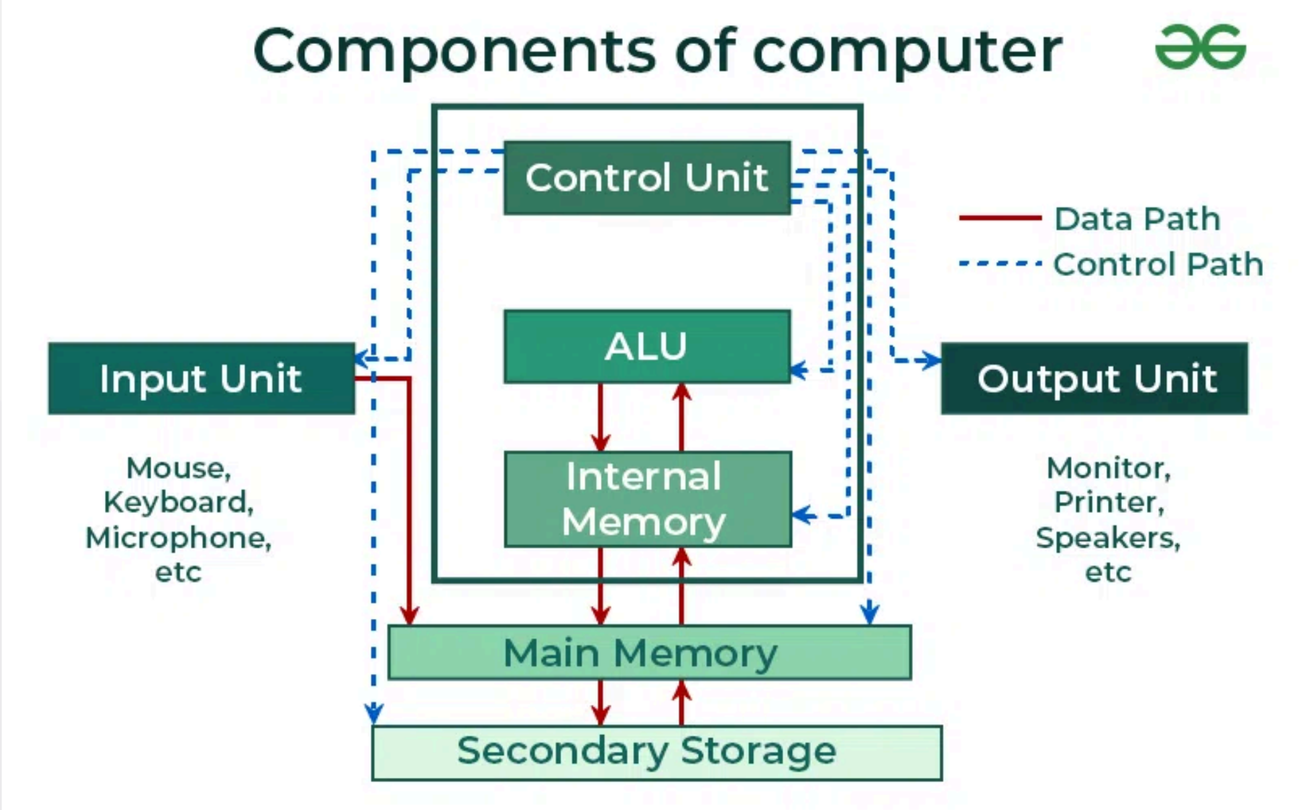
A computer system and is composed of two parts: hardware (the physical elements) and software (the instructions that tell the hardware what it has to do, how to solve the problem). Sometimes, some authors include two parts more: the user and the data.

# ACTIVITY: COMPUTER HISTORY

Write a short article about the origins of computer science and the development of the first computers. Include references to early machines such as the Pascaline, Colossus, ENIAC, and the Intel 8086, among others.

# COMPUTER ARCHITECTURE

Computer architecture is a theoretical model that describes the structure, components, and organization of a computer system



# VON NEUMANN ARCHITECTURE

Von Neumann architecture is an abstract model created in 1946 that describes a computer architecture.

In Von Neumann architecture, the main elements of a computer are:

- CPU (Central Process Unit).
- Memory unit.
- Buses.
- I/O (Input/Output to external devices).

# VON NEUMANN ARCHITECTURE (II)

Von Neumann architecture works as described:

- Components areconnected by buses.
- It can execute elemental instructions (machine code).
- The instructions are stored in the memory unit. Also, the data has to be stored in memory unit.
- Both instructions and data could be provided by input devices or generated by other programs (compiler, linker, etc.).
- The CPU reads an instruction and the control unit generates each signal to perform the instruction.
    - If it is required, CPU can perform arithmetic-logic operations.

- The CPU results can be stored on the memory unit, or it can be sent to output devices.

# VON NEUMANN ARCHITECTURE (III)

Over the years, processors and memory have advanced dramatically to meet growing computational needs → Moore's law

Modern CPUs achieve remarkable performance, and memory systems provide massive storage capacity.

This evolution has led to highly specialized components, enabling microprocessors to integrate billions of transistors on a single chip.

Nowadays, there are several improvements of this architecture, including features like mapped I/O (memory unit and I/O devices are treated like the same memory).

If you want to try how Von Neumann architecture works, you can use this website: https://lab.xitrus.es/VonNeumann/

# VON NEUMANN ARCHITECTURE (III)

Over the years, processors and memory have advanced dramatically to meet growing computational needs → Moore's law

Modern CPUs achieve remarkable performance, and memory systems provide massive storage capacity.

This evolution has led to highly specialized components, enabling microprocessors to integrate billions of transistors on a single chip.

Nowadays, there are several improvements of this architecture, including features like mapped I/O (memory unit and I/O devices are treated like the same memory).

If you want to try how Von Neumann architecture works, you can use this website: https://lab.xitrus.es/VonNeumann/

# HARVARD ARCHITECTURE

Harvard's architecture is other abstract model that describes a computer architecture. It is less implemented than Von Neumann architecture.

The main differences between Harvard architecture and Von Neumann architecture are:

- In Von Neumann architecture, data and instructions are stored on the same memory. CPU only can read an instruction or a data at the same time. In Harvard architecture, data and instructions are stored separately, and each one has its own bus. CPU can read both at the same time (an instruction and a data).

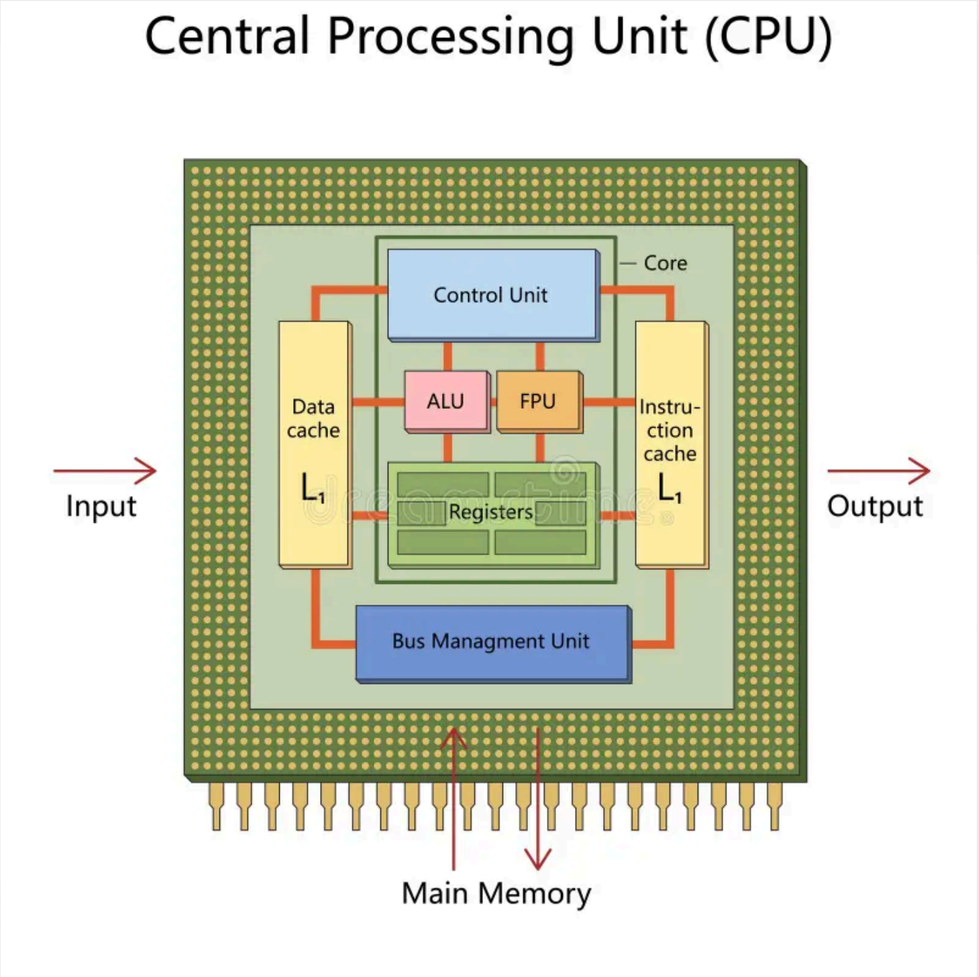
- In Von Neumann architecture, data and instructions are stored on the same memory, and they share address space. In Harvard architecture, they are stored in different memories.

# HARVARD ARCHITECTURE (II)



# IMPORTANT

- Usually, real computer don’t use pure Von Neumann architecture or pure Harvard architecture. The only are abstract models.



# FUNCTIONAL ELEMENTS OF A COMPUTER

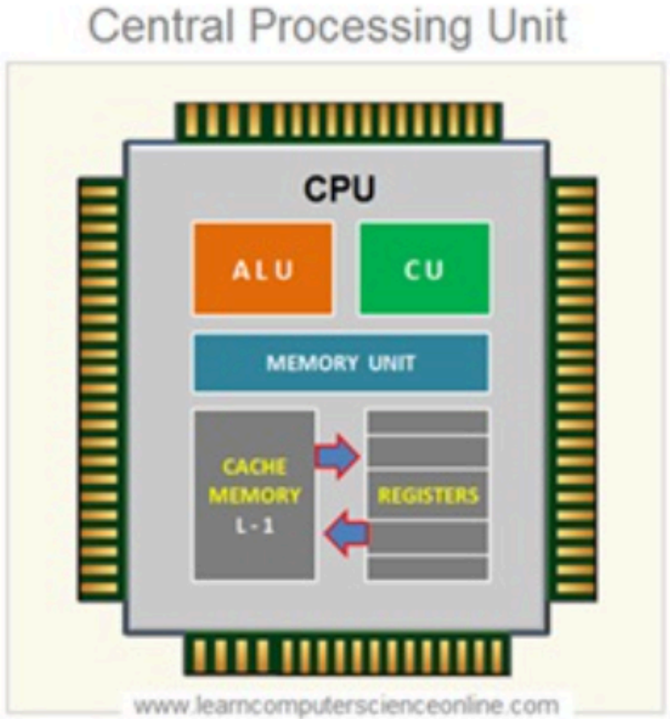
In this point, we are going to describe functional elements of a computer that implements Von Neumann architecture.

# CPU(CENTRAL PROCESS UNIT)

The CPU is the core of the computer. CPU read instructions from memory and execute them, doing the required operations. It is composed of:

- **Registers:** they are little and fast memories. They are used as temporal storage when an instruction is being executed or to store temporally results of operations. The most important registers are:
    - **Program counter (PC):** an incremental counter that stores the memory address of the next instruction that is going to be executed.
    - **Memory address register (MAR):** it stores the address of a block of memory for reading from or writing to.

# CPU REGISTERS




CPU Registers

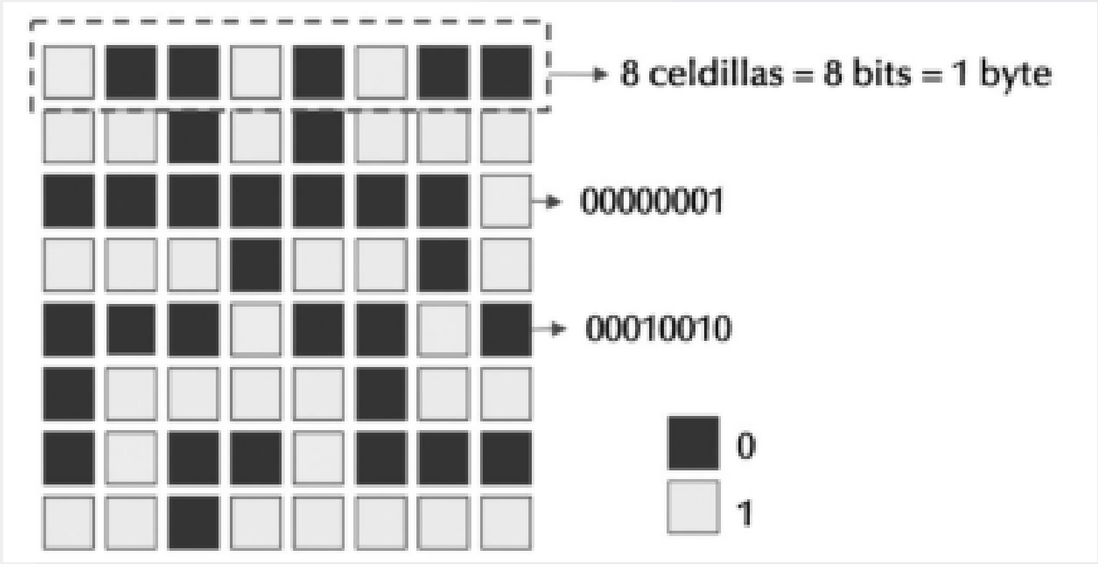
Special Purpose Registers

General Purpose Registers

The CPU Registers are High Speed memory area inside the processor chip And used by the processor during the program execution.

# CPU REGISTERS (II)

- The memory inside the CPU (registers) is made up of a series of **cells** or **bistables** (so called because they can only hold two possible states)
- Each one stores a single bit.
- Access to a group of these cells is generally performed through an address.
- These cells can be accessed in both read and write modes.



# CPU REGISTERS (III)

- The cells are the basic information units, called bits. These bits can take two possible values, 1 and 0 (presence or absence of electrical energy).

- These cells are used to store the machine’s data and instructions. These instructions are grouped into what are known as programs. Programs are sets of instructions stored sequentially, and their instructions are executed one after another. The system executes these programs, which are located in main memory.

- The data used by the programs, as mentioned, also reside in main memory, and the system must fetch the instructions, interpret them, and access the data whenever necessary, either in write or read mode.

# CPU (CENTRAL PROCESS UNIT) (II)

- **Registers (II):**
  - **Memory data register (MDR):** a two-way register that stores data fetched from memory (and ready for the CPU to process) or data waiting to be stored in memory.
  - **Instruction register (IR):** it is a temporal register to store the instruction that has just been fetched from memory.
  - **General purpose registers:** those registers are used to store information temporally, for example to store operands of an addition or to store the result of an arithmetic-logic operation.
  - **Flags:** These are very simple records. They can only be in two states: “On” or “Off”. They are used to indicate the status of an operation. Example: flag Z is set to 1 if the operation has resulted in 0, N is set to 1 if the result is negative.

# CPU (CENTRAL PROCESS UNIT) (III)

- **Internal bus:** is an internal bus that connect registers, control unit and arithmetic-logic unit.
- **Control unit (UC):** it decides what to do in each moment. Its components are:
  - **Decoder:** its function is to decode instructions and prepare all the signals to execute the program properly.
    - For instance, an instruction can be “ADD” (which adds up two numbers that are in memory). When it comes to executing, the decoder generates a lot of small signals such as reading the first number of the memory, taking it to the ULA, reading the second number, taking it to the ULA, perform the operation (adding) and saving the result in memory. In this case, an instruction has generated six signals 1.
  - **Clock:** is the system that sets the pace at which things are done. It's like the conductor's baton. It allows all system components to operate perfectly together.

# CPU (CENTRAL PROCESS UNIT) (IV)

- **Arithmetic-Logic Unit (ULA):** it is a unit designed to perform arithmetic operations (addition, subtraction, multiplication, etc.) and logic operations (not, and, or, ...). It's the worker.

To execute an instruction, the CPU follow those steps:

- Use the Program Counter to fetch the new instruction from memory.
- Decode the instruction and fetch data from memory if it is required.
- Perform the instruction and store data in an internal register or memory.

The instruction cycle will be detailed in later in the section Instruction cycle.

to measure the speed of a CPU, we use frequency units that indicate the number of times a thing repeats itself per unit of time. In general, we use the Hz (or its multiples) that indicates the number of operations that can be performed in a second.

# CPU (CENTRAL PROCESS UNIT) (V)

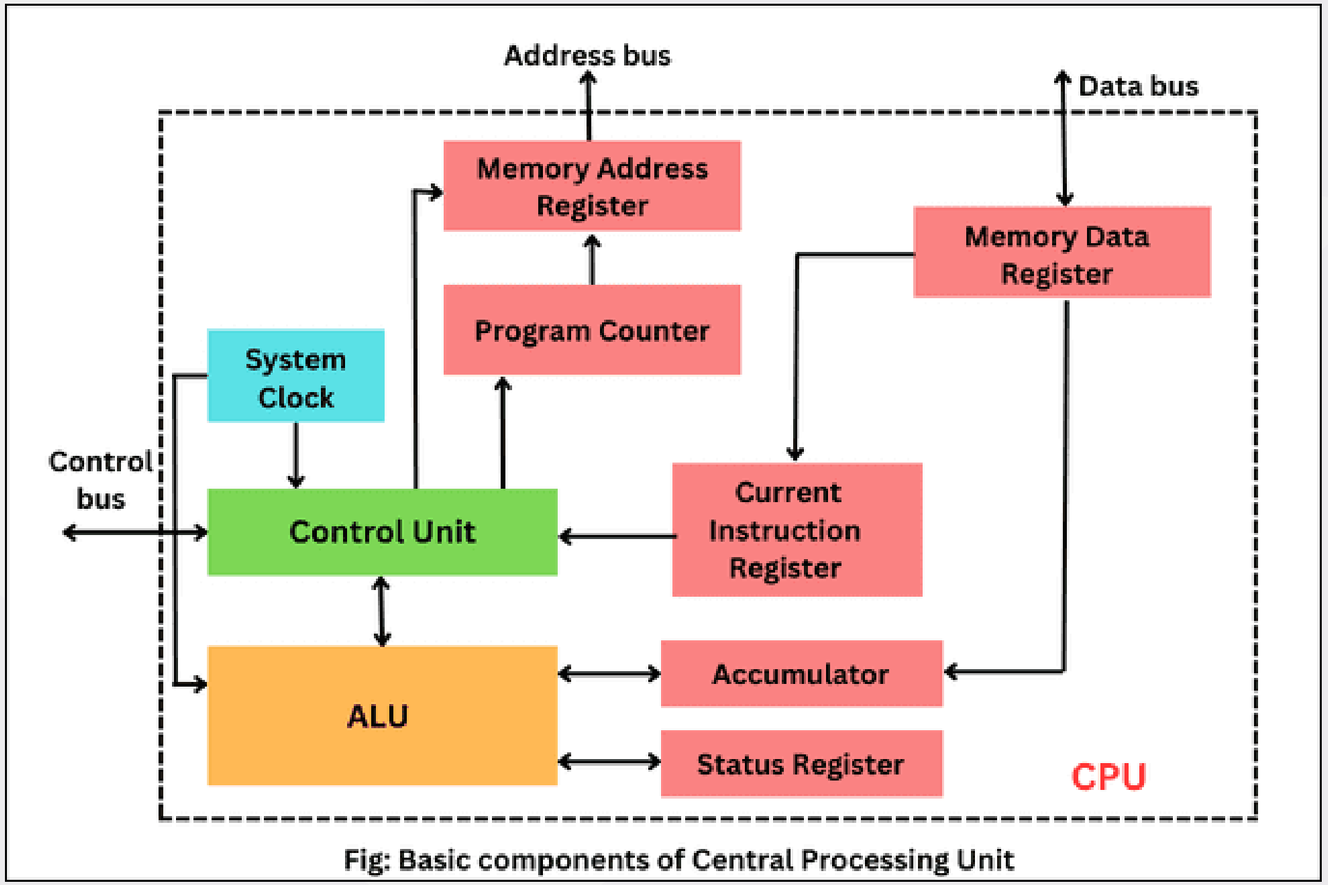
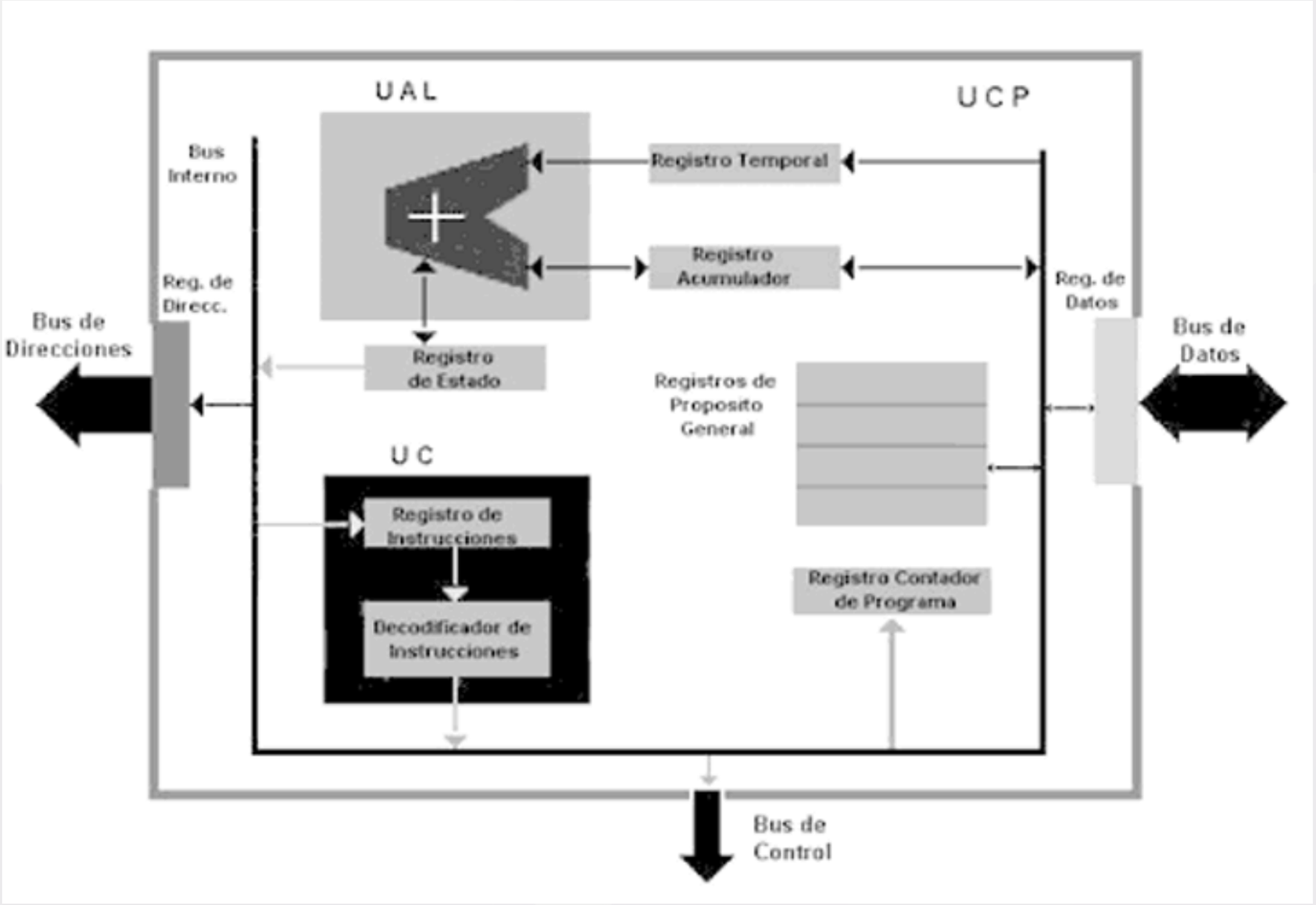
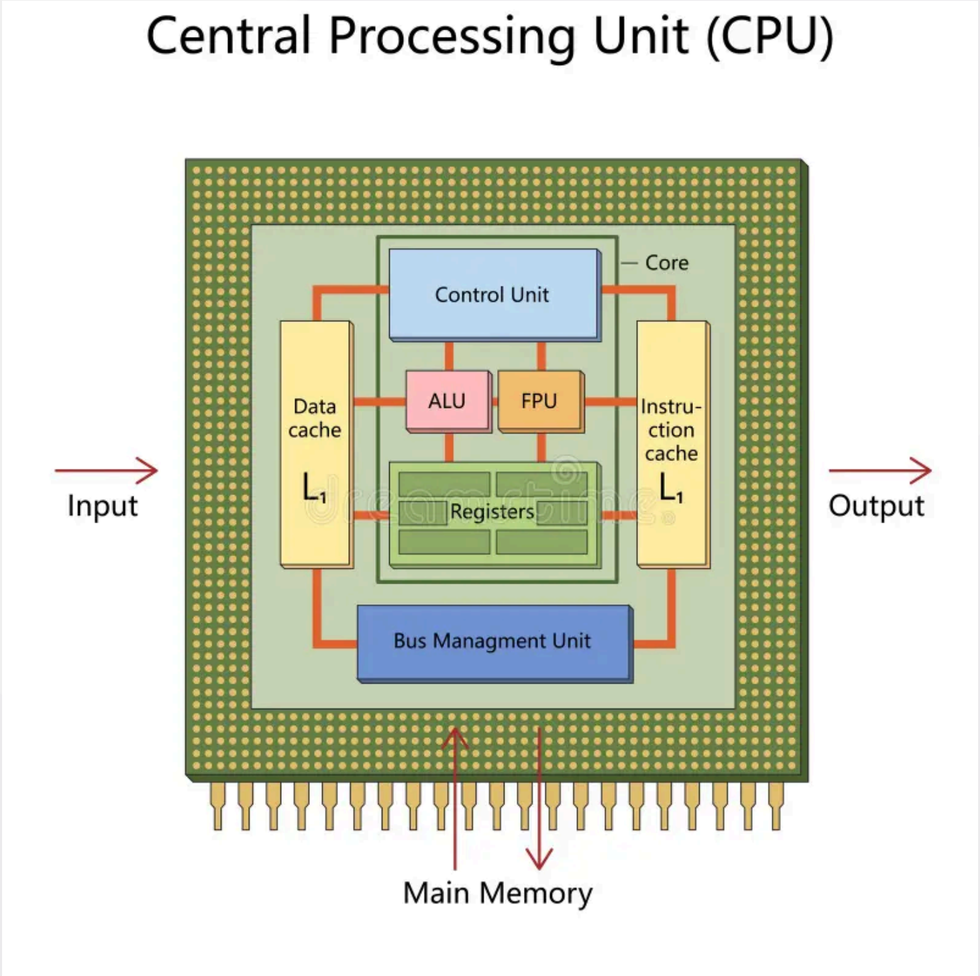



Fig: Basic components of Central Processing Unit

# CPU(CENTRAL PROCESS UNIT) (VI)



# CPU(CENTRAL PROCESS UNIT) (VII)

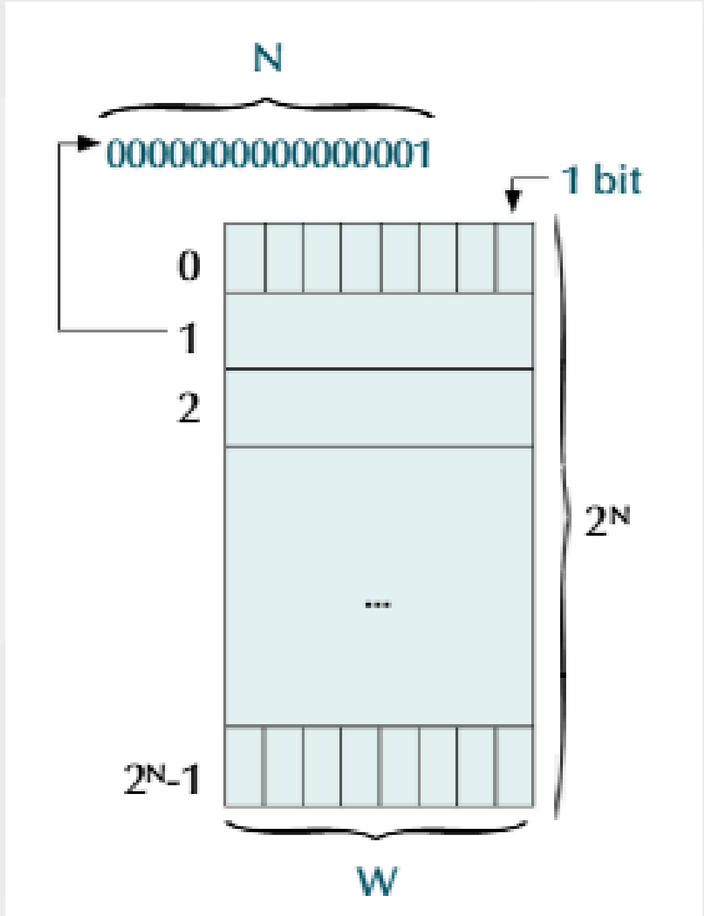


# RAM MEMORY: BASIC STRUCTURE

- RAM (Random Access Memory) consists of a series of memory cells, each containing several bits.
- Each bit stored in a cell can hold one of the following values: 0, 1, or no value.
- Typically, each **cell** contains one **byte** (1 byte = 8 bits).
- RAM is used as temporary storage, allowing fast access to data needed by the CPU.
- All these cells have a specific address, and the cell is the minimum unit of memory, which can be fully read or written.

# RAM MEMORY: BASIC STRUCTURE (II)

- Memory consists of cells numbered 0, 1, …, $2^N$ −1, each with a binary address.
- If the system uses two bytes (16 bits) to address memory, cell 1 is represented as 0000000000000001, and accessing that cell requires specifying its full binary address.
- It must also be considered that the number of bits used to address a memory cell directly determines the **total memory capacity** of the system.



# RAM MEMORY: BASIC STRUCTURE (III)

It is important to distinguish between a memory cell’s address and its content:

- **Address of a memory cell:** The address is expressed as a binary number (typically two bytes) and indicates where the data is located in memory.

- **Content of a memory cell:** The address points to a cell that contains a value (in this example, 1 byte = 8 bits).

# PROGRAM EXECUTION AND MEMORY ACCESS

- The time the system takes to read or write any memory cell is the same, regardless of its position.
- Programs consist of a sequence of instructions executed by the processor, and to be executed, they must reside in memory.
- The data used by these programs must also be stored in memory so the CPU can access them when needed.
- **Memory access time** is the amount of time required to perform a read or write operation, typically measured in nanoseconds — where one nanosecond is one billionth of a second.

# MEMORY OPERATIONS: READ AND WRITE

**a) Read (address):**

- Reading from a memory address means retrieving the content stored in a specific memory cell.
- The read operation does not modify the value of the cell; the data remains unchanged.
- It is a non-destructive operation, so it can be repeated without altering the stored information.

# MEMORY OPERATIONS: READ AND WRITE (II)

**b) Write (address, value):**

- Writing to a memory address means storing a value into a specific memory cell.
- The write operation modifies the cell’s content by copying data from a register into that memory location.
- It is a destructive operation in the sense that it replaces the previous value with a new one.

# MEMORY OPERATIONS: READ AND WRITE (III)

To perform memory operations, the system must send either a read signal or a write signal to the memory module.

Two essential registers are required for these operations: MAR (Memory Address Register) and MDR (Memory Data Register)

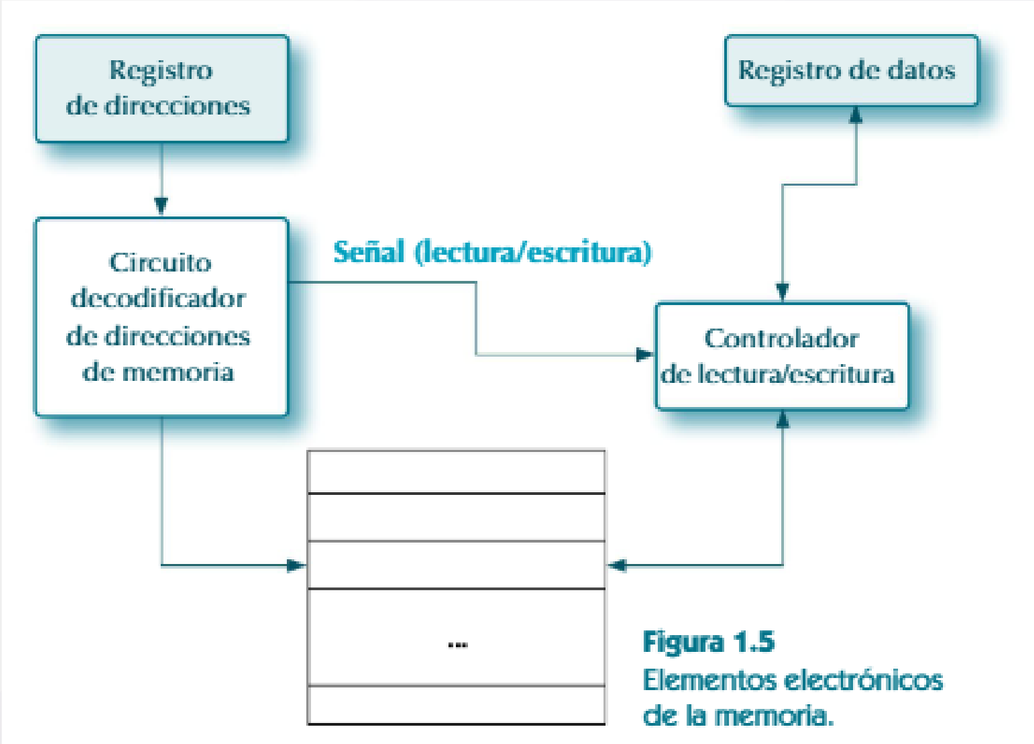
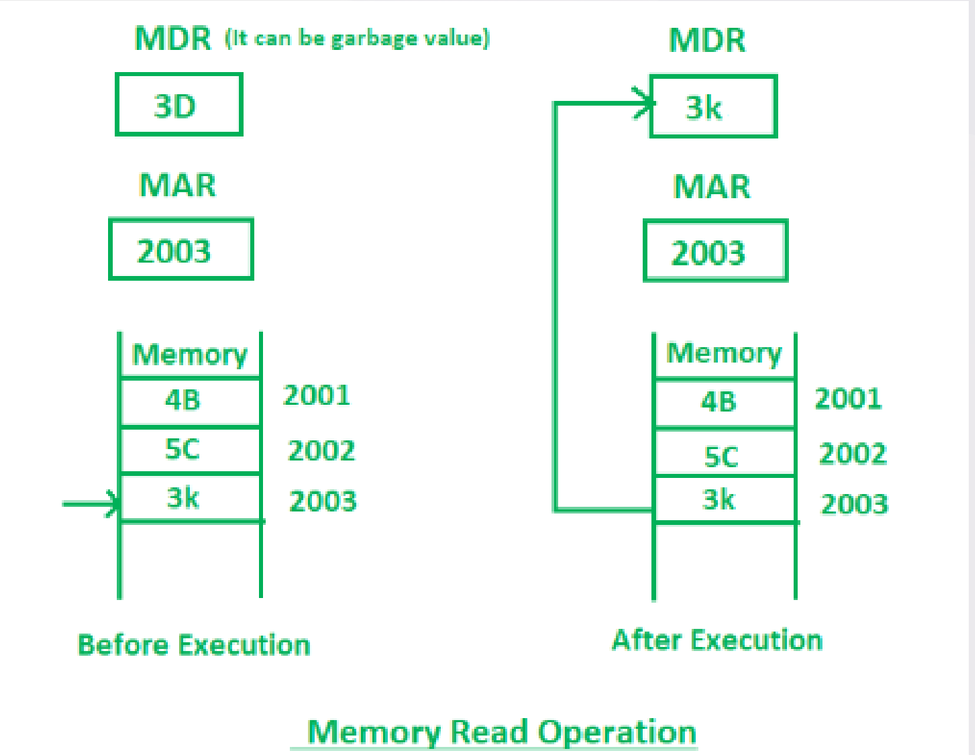



**Figura 1.5**
Elementos electrónicos de la memoria.

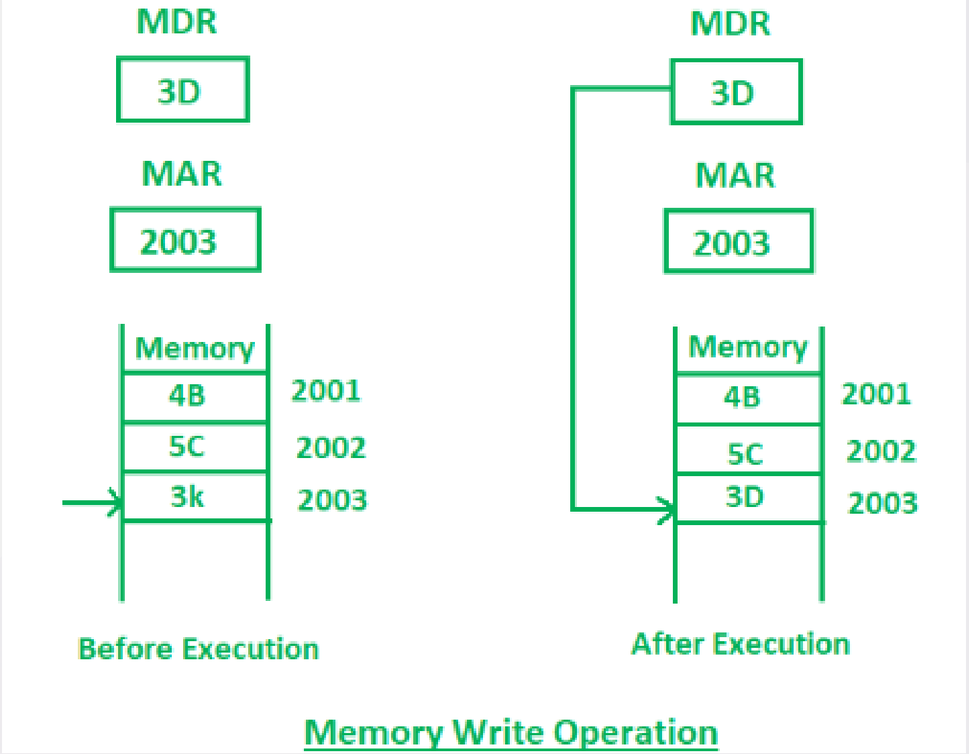
# MEMORY OPERATIONS: READ

- The address is loaded into the address register.
- The address is decoded, locating the specific memory cell to be accessed.
- The content of that memory cell is copied into the data register.
- The value stored in memory remains unchanged, since reading is a non-destructive operation.



# MEMORY OPERATIONS: WRITE

- The address is loaded into the address register.
- The value to be stored is placed in the data register.
- The address is decoded, locating the specific memory cell where the data will be written.
- The content of the memory cell is replaced with the value from the data register.
- Writing is a destructive operation, meaning the previous value in that cell is overwritten.



Memory Write Operation

# INPUT/OUTPUT SUBSYSTEM

- The input/output (I/O) subsystem controls the devices that allow the computer system to communicate with the external environment.
- These devices include: Display (monitor), Keyboard, Printer, Mouse and other peripherals that enable user interaction.
- The I/O subsystem also manages secondary storage devices, such as hard drives, solid-state drives, or external storage units, allowing the system to store and retrieve information beyond main memory.
- It acts as a bridge between the CPU and the outside world, ensuring smooth data exchange and device coordination.

# MEMORY AND STORAGE ACCESS TIME

- The time required for memory to perform a read or write operation is usually measured in nanoseconds (ns).

- In contrast, hard drives and other secondary storage devices measure access time in milliseconds (ms).

- This difference shows that main memory (RAM) is millions of times faster than secondary storage, which is why data must be loaded into memory before the CPU can process it efficiently.

- Example comparison:
  - RAM: ~10–100 nanoseconds
  - Hard drive: ~5–10 milliseconds
  - SSD: ~0.1–1 millisecond

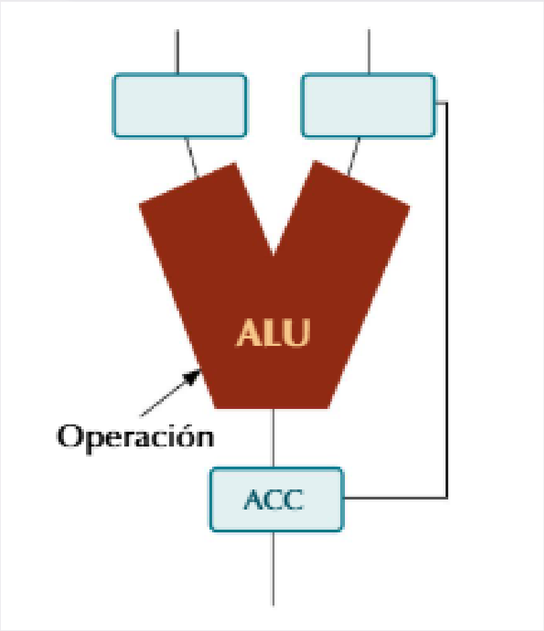
# THE ALU (ARITHMETIC LOGIC UNIT)

- It is an integrated circuit responsible for performing arithmetic operations (+, −, ×, ÷) and logical operations (=, <, >, AND, OR, NOT...).

- It consists of:
  - A set of circuits that execute arithmetic and logical operations.
  - A series of registers that store intermediate results during computation.
  - A bus that interconnects the registers with the operation circuitry.

- The ALU can take up to two operands at a time.

- The result of the operation is typically stored in a special register called the accumulator (ACC).



# THE CONTROL UNIT (CU)

- It is responsible for executing program instructions, one by one.
- Programs are stored in main memory as binary machine instructions, the only language the hardware understands.
- The CU fetches each instruction from memory, interprets it, and coordinates the actions of the CPU components (ALU, registers, buses...).
- It ensures that every step of the instruction cycle (fetch → decode → execute) happens in the correct order.
- The CU acts as the “conductor” of the computer system, directing data flow and timing so operations occur smoothly and correctly.

# CPU FETCH-EXECUTE CYCLE

1. **Fetch** the instruction from memory

   - The CU reads the next instruction stored in main memory.
   - The address of the instruction is typically held in the Program Counter (PC).

2. **Decode** the instruction

   - The CU interprets the binary pattern to determine which operation must be performed.
   - It identifies the required operands, registers, and functional units.

# CPU FETCH-EXECUTE CYCLE (II)

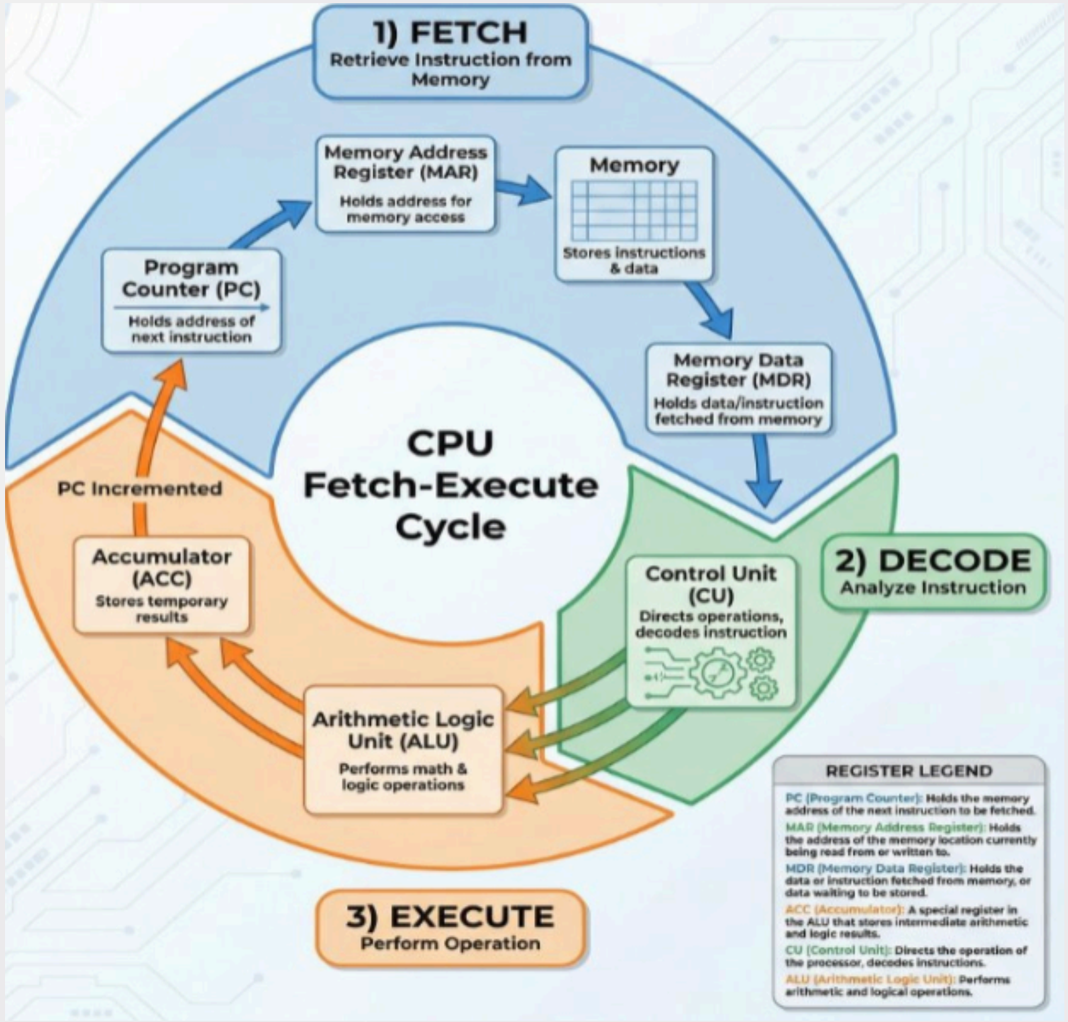
3. **Execute** the instruction

- The CU sends the appropriate control signals to:
  - ALU (for arithmetic/logic operations)
  - Memory (for read/write operations)
  - I/O subsystem (for input/output operations)
- Each component performs its part of the instruction.

4. **Continue** execution until HALT

- The CU repeats the cycle (fetch → decode → execute) for the next instruction.
- The process continues until a HALT instruction is encountered, signaling the end of the program.

# CPU FETCH-EXECUTE CYCLE (III)



# STRUCTURE OF A MACHINE INSTRUCTION

- A machine-language instruction is composed of two main fields:
  - Operation code (**opcode**)
  - Values or memory addresses containing the operands needed for the operation

- Example: “SUM A + B”
  - Suppose the instruction is SUM A + B.
  - The opcode for the SUM operation is 105, which in binary is 1101001.
  - The operands A and B correspond to the values 7291 and 15484, which also are represented in binary.



# PRACTICAL CASE: OUR HW1 MACHINE

**1. Create a program using HW1 machine instructions that reads two values from the keyboard and displays their sum on the screen**

```
READ RA
READ RB                 // We read the numbers to add in registers A and B
MOVE ACC RB             // We move into ACC operand B
SUM ACC RA              // We sum ACC that contains B with RA that contains A
MOVE RC ACC             // We take the result to register C
PRINT “THE SUM IS:”
PRINT RC                // We print the result on the screen
```

# PRACTICAL CASE: OUR HW1 MACHINE

**2. This time, create a program using HW1 machine instructions that reads three values from the keyboard and displays their sum on the screen.**

```
READ RA
READ RB
READ RC                 // We read the numbers to add in registers A, B and C
MOVE ACC RB             // We move into ACC operand B
SUM ACC RA              // We sum ACC that contains B with RA that contains A
SUM ACC RC              // We sum ACC that contains A+B with RC that contains C
MOVE RD ACC             // We take the result to register D
PRINT “THE SUM IS:”
PRINT RD                // We print the result on the screen
```

# PRACTICAL CASE: OUR HW1 MACHINE

**3. Create a program using HW1 machine instructions that displays the numbers from 1 to 10 on the screen.**

```
MOVE ACC 11       // A counter stored in A, when that reachs 11 program ends
MOVE RB ACC
MOVE ACC 1
MOVE RA ACC                 // We initialize RA with 1
LOOP:                       // INTX
PRINT RA
MOVE ACC 1
MOVE RA ACC
MOVE ACC RB
SUB RA ACC                  // We substract 11. If this is zero program must end
JUMP IF NOT ZERO LOOP       // It could also be 5
PRINT “End of program”
```

# ¿ANY QUESTION?